

|                                                                                   |                                          |                  |
|-----------------------------------------------------------------------------------|------------------------------------------|------------------|
|  | SR UNIVERSITY                            |                  |
|                                                                                   | Campus Warangal                          |                  |
|                                                                                   | Program: II - B. Tech (CS& AI) Batch:    |                  |
|                                                                                   | Name: ABHISHEK H YENDAY<br>No:2503A52479 | Reg.             |
|                                                                                   | Department: Computer Science and AI      | Semester: I      |
|                                                                                   | AI Assisted Coding-Assignment            | Date: 06-10-2026 |

## TASK-1

### 1. Original code:

```

a = 80
b = 70
c = 90
x = (a + b + c) / 3
if x >= 50:
    print("pass")
else:
    print("fail")

```

### 2.Prompt:

Rewrite this code with meaningful variable names, a calculate\_average() function, and clear formatting. Keep its behavior the same and verify both versions produce the same result.

### 3.AI suggestion:

Store the marks in clearly named variables, calculate their average in a function, and use a descriptive average variable for the pass/fail check.

#### 4.Refactored code:

```
from contextlib import redirect_stdout
from io import StringIO
```

```
def run_original_version():
```

```
    a = 80
```

```
    b = 70
```

```
    c = 90
```

```
    x = (a + b + c) / 3
```

```
    if x >= 50:
```

```
        print("pass")
```

```
    else:
```

```
        print("fail")
```

```
def calculate_average(mark_one, mark_two, mark_three):
```

```
    return (mark_one + mark_two + mark_three) / 3
```

```
def run_refactored_version():
```

```
    first_mark = 80
```

```
    second_mark = 70
```

```
    third_mark = 90
```

```
    average_mark = calculate_average(first_mark, second_mark, third_mark)
```

```
    if average_mark >= 50:
```

```

        print("pass")
    else:
        print("fail")

def capture_output(function):
    output = StringIO()
    with redirect_stdout(output):
        function()
    return output.getvalue().strip()

if __name__ == "__main__":
    original_result = capture_output(run_original_version)
    refactored_result = capture_output(run_refactored_version)

    assert original_result == refactored_result

    print(f"Original result: {original_result}")
    print(f"Refactored result: {refactored_result}")
    print("Verified: both versions produce the same result.")

```

4.output ; Original result: pass

Refactored result: pass

Verified: both versions produce the same result.

5. explanation :

- ☐ In the **original version**, the marks are 80, 70, 90.
- ☐ The average is **80**, so the result is pass.

- ❑ In the **refactored version**, the same marks are passed to the reusable `calculate_average()` function.
- ❑ The function calculates the same average, **80**, so the result is also pass.
- ❑ `capture_output()` temporarily captures the printed output from both functions.
- ❑ The assert statement checks whether both results are exactly the same.
- ❑ Since both results are "pass", the assertion succeeds.
- ❑ Finally, the program confirms that both versions produce the same result.

## TASK – 2

### 1.Original code:

```
name1 = "Arun"
```

```
marks1 = 85
```

```
if marks1 >= 50:
```

```
    print(name1, "Pass")
```

```
else:
```

```
    print(name1, "Fail")
```

```
name2 = "Priya"
```

```
marks2 = 72
```

```
if marks2 >= 50:
```

```
    print(name2, "Pass")
```

```
else:
```

```
    print(name2, "Fail")
```

### 2.prompt:

Identify the duplicated logic in the given student pass/fail program. Refactor the repeated logic into a reusable function. Test the function with at least four students, verify that the output remains correct, and explain how removing code duplication improves maintainability.

3. AI suggestion;

The if-else logic for checking whether a student has passed is repeated for every student. Create a reusable function such as `check_result(name, marks)` and call it for each student. This avoids writing the same logic multiple times and makes future changes easier.

4. refactored code:

```
def check_result(name, marks):
```

```
    if marks >= 50:
```

```
        print(name, "Pass")
```

```
    else:
```

```
        print(name, "Fail")
```

```
# Testing with four students
```

```
check_result("Arun", 85)
```

```
check_result("Priya", 72)
```

```
check_result("Rahul", 45)
```

```
check_result("Sneha", 38)
```

5. output-

Arun Pass

Priya Pass

Rahul Fail

Sneha Fail

6.explanation- The original code repeats the same pass/fail logic for each student. The refactored code places this logic inside the `check_result()` function and reuses it for different students.

### TASK 3

#### 1.original code:

Legacy code:

```
age = 68
```

```
temperature = 39
```

```
if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 and temperature < 38:
    print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

#### 2. Prompt:

Rewrite this priority-checking code using meaningful variable names and a `classify_priority()` function. Keep its behavior unchanged, format it clearly, and verify both versions produce the same result.

#### 3.AI suggestion:

Use descriptive names for the patient's age and temperature, and put the decision rules in a function. Check the combined high-priority condition first, then handle cases where only one threshold is reached.

#### 4.refactored code-

```
from contextlib import redirect_stdout
from io import StringIO
```

```
def run_original_priority_check():
```

```
    age = 68
```

```
    temperature = 39
```

```
    if age >= 60 and temperature >= 38:
```

```
        print("High Priority")
```

```
    elif age >= 60 and temperature < 38:
```

```
        print("Medium Priority")
```

```
    elif age < 60 and temperature >= 38:
```

```
        print("Medium Priority")
```

```
    else:
```

```
        print("Normal Priority")
```

```
def classify_priority(patient_age, body_temperature):
```

```
    is_senior = patient_age >= 60
```

```
    has_high_temperature = body_temperature >= 38
```

```
    if is_senior and has_high_temperature:
```

```
        return "High Priority"
```

```
    if is_senior or has_high_temperature:
```

```
        return "Medium Priority"
```

```
    return "Normal Priority"
```

```
def run_refactored_priority_check():
```

```
patient_age = 68
body_temperature = 39
print(classify_priority(patient_age, body_temperature))
```

```
def capture_priority_output(function):
```

```
    output = StringIO()
```

```
    with redirect_stdout(output):
```

```
        function()
```

```
    return output.getvalue().strip()
```

```
if __name__ == "__main__":
```

```
    original_result = capture_priority_output(run_original_priority_check)
```

```
    refactored_result = capture_priority_output(run_refactored_priority_check)
```

```
    assert original_result == refactored_result
```

```
    print(f"Original result: {original_result}")
```

```
    print(f"Refactored result: {refactored_result}")
```

```
    print("Verified: both versions produce the same result.")
```

5.output- Original result: High Priority

Refactored result: High Priority

Verified: both versions produce the same result.

6.explanation- Descriptive names show what the values represent, and the function makes

# the classification reusable. The combined conditions avoid repeating

# similar branches while preserving the original priority rules.

#### TASK – 4

##### 1 ORIGINAL CODE-

```
patient_ids = [101, 205, 309, 412, 518, 623]
```

```
target = 412
```

```
found = False
```

```
for patient_id in patient_ids:
```

```
    if patient_id == target:
```

```
        found = True
```

```
if found:
```

```
    print("Patient Found")
```

```
else:
```

```
    print("Patient Not Found")
```

##### 2.PROMPT-

"Here is a Python snippet that checks whether a patient ID exists in a list using a loop and a flag variable. How can I make it more readable and Pythonic?"

### 3.AI SUGGESTION-

**Readability:** Replace the loop and flag with the in operator, wrap the check in a small function with a descriptive name, and use constants and f-strings for clearer output.

### 4.REFACTORED CODE-

```
PATIENT_IDS = {101, 205, 309, 412, 518, 623} # set: fast membership lookup
```

```
def is_patient_registered(patient_id, registered_ids=PATIENT_IDS):
```

```
    """Return True if the patient ID exists in the registered set."""
```

```
    return patient_id in registered_ids
```

```
def report_patient_status(patient_id):
```

```
    """Print whether the given patient ID was found."""
```

```
    if is_patient_registered(patient_id):
```

```
        print(f"Patient {patient_id}: Patient Found")
```

```
    else:
```

```
        print(f"Patient {patient_id}: Patient Not Found")
```

```
# Tests: existing IDs, missing IDs, and edge cases
```

```
test_ids = [412, 101, 623, 999, 0, -5]
```

```
for test_id in test_ids:
```

```
    report_patient_status(test_id)
```

5.OUTPUT- Patient 412: Patient Found

Patient 101: Patient Found

Patient 623: Patient Found

Patient 999: Patient Not Found

Patient 0: Patient Not Found

Patient -5: Patient Not Found

## 6.EXPLANATION-

1. The loop and found flag became a single patient\_id in registered\_ids expression.
2. The logic moved into named functions, so it is reusable and testable.
3. The list became a set, which makes lookups much faster at scale.

## TASK -5

### 1.ORIGINAL CODE-

Legacy code:

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]
```

```
total = 0
```

```
for s in students:
```

```
    total = total + s[1]
```

```
avg = total / len(students)
```

```
for s in students:
```

```
    if s[1] >= 50:
```

```
        print(s[0], "Pass")
```

```
    else:
```

```
        print(s[0], "Fail")
```

```
print("Average =", avg)
```

### 2.PROMPT -

"Review this legacy Python code that calculates a class average and prints Pass/Fail for each student. Identify at least five improvements covering readability, naming, reusability, correctness, and Pythonic style. For each one, explain why it matters and show a short example."

### 3.AI SUGGESTION-

|                          |                 |
|--------------------------|-----------------|
| Use tuple unpacking      | <b>Accepted</b> |
| 2 Use sum()              | <b>Accepted</b> |
| 3 Extract functions      | <b>Accepted</b> |
| 4 Add PASS_MARK constant | <b>Accepted</b> |
| 5 Use meaningful names   | <b>Accepted</b> |

#### 4. REFACTORED CODE-

```
PASS_MARK = 50
```

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]
```

```
def calculate_average(records):
```

```
    """Return the average marks, or 0 if there are no records."""
```

```
    if not records:
```

```
        return 0
```

```
    total_marks = sum(marks for _, marks in records)
```

```
    return total_marks / len(records)
```

```
def get_result(marks, pass_mark=PASS_MARK):
```

```
    """Return 'Pass' or 'Fail' for the given marks."""
```

```
    return "Pass" if marks >= pass_mark else "Fail"
```

```

def print_report(records):
    """Print each student's result followed by the class average."""
    for name, marks in records:
        print(f"{name} {get_result(marks)}")
    print("Average =", calculate_average(records))

if __name__ == "__main__":
    print_report(students)

```

## 5.OUTPUT-

Arun Pass

Priya Fail

Rahul Pass

Meena Pass

Average = 65.75

## 6.EXPLANATION-

1. Tuple unpacking replaced s[0] and s[1].
2. sum() with a generator expression replaced the manual accumulator.
3. Three single-purpose functions replaced top-level script code.
4. PASS\_MARK replaced the hard-coded 50.
5. Descriptive names replaced s, avg, and total.
6. An empty-list guard was added, and \_\_main\_\_ protects imports

